Python Programming for Beginners

Part 6 - Functions in Python

Topics Covered

- What are Functions?
- Why Do We Need Functions?
- Advantages of Functions
- Types of Functions
- Defining a Function
- Calling a Function
- Function Parameters
- Function Arguments
- Default Arguments
- Keyword Arguments
- Arbitrary Arguments (`*args`)
- Arbitrary Keyword Arguments (`**kwargs`)
- Return Statement
- Scope of Variables
- Recursion (Introduction)
- Lambda Functions (Introduction)

Learning Objectives

By the end of this lesson, you will be able to:

- Understand what functions are and why they are used.
- Create and call user-defined functions.
- Pass data to functions using parameters and arguments.
- Use different types of function arguments.
- Return values from functions using the `return` statement.
- Understand the difference between local and global variables.
- Write simple recursive functions.
- Create basic lambda functions.
- Write clean, reusable, and organized Python programs.

What are Functions?

A **function** is a reusable block of code that performs a specific task.

Instead of writing the same code multiple times, you can write it once inside a function and call it whenever you need it.

Functions help make programs **organized, reusable, and easier to maintain**.

Real-Life Example

Think of a **coffee machine**.

- Press the **Coffee** button → It makes coffee.
- Press the **Tea** button → It makes tea.
- Press the **Hot Water** button → It dispenses hot water.

Each button performs a specific task, just like a **function** in Python.

Syntax

```
def function_name():
    # Code to execute
```

Syntax Explanation

- `def` → Keyword used to define a function.
- `function_name` → Name of the function.
- `()` → Parentheses that can contain parameters.
- `:` → Marks the beginning of the function body.
- Indented block → Contains the statements that belong to the function.

Example 1: Creating a Function

In [ ]:
```
def greet():
    print("Welcome to Python!")
```

This code creates a function named `greet()`. The function is created, but it is **not executed** until it is called.

Example 2: Creating Another Function

In [ ]:
```
def show_message():
    print("Learning Python is fun!")
```

The function `show_message()` is also created but does not produce any output until it is called.

Key Points

- A function is a reusable block of code.
- Functions perform a specific task.
- Functions help reduce code duplication.
- Functions improve code readability and organization.
- A function must be called to execute its code.

Why Do We Need Functions?

As programs become larger, writing the same code repeatedly makes them difficult to read, maintain, and update.

Functions solve this problem by allowing us to write a block of code **once** and use it **multiple times**.

They help make programs **shorter, cleaner, and more organized**.

Without Functions

In [ ]:
```
print("Welcome to Python!")
print("Welcome to Python!")
print("Welcome to Python!")
```

The same statement is written multiple times.

With Functions

In [ ]:
```
def greet():
    print("Welcome to Python!")

greet()
greet()
greet()
```

Output

```
Welcome to Python!
Welcome to Python!
Welcome to Python!
```

In this example, the function is written once and called three times.

Why Use Functions?

Functions help us:

- Reduce code duplication.
- Reuse code whenever needed.
- Make programs easier to read.
- Improve code organization.
- Simplify debugging and maintenance.
- Break large programs into smaller, manageable parts.

Real-Life Example

Imagine writing your name **100 times** on paper.

Instead of writing it manually every time, you could use a **stamp**.

A function works like a stamp—you create it once and use it whenever needed.

Key Points

- Functions eliminate repetitive code.
- A function can be called multiple times.
- Functions improve readability and maintainability.
- Large programs become easier to organize using functions.

Advantages of Functions

Functions provide several benefits that make programs easier to write, understand, and maintain.

---

1. Code Reusability

A function can be written once and called multiple times, reducing the need to write the same code repeatedly.

---

2. Reduces Code Duplication

Functions eliminate repetitive code, making programs shorter and more efficient.

---

3. Improves Readability

Breaking a program into small functions makes the code easier to read and understand.

---

4. Easier to Maintain

If a change is needed, you only need to update the function instead of modifying the same code in multiple places.

---

5. Simplifies Debugging

Errors can be found and fixed more easily because each function performs a specific task.

---

6. Makes Programs Modular

Functions divide a large program into smaller, manageable parts, making development easier.

---

7. Saves Time

Functions allow you to reuse existing code instead of writing it again, saving development time.

Summary Table

Advantage	Description
Code Reusability	Write once, use many times
Reduces Code Duplication	Avoids repeating the same code
Improves Readability	Makes code easier to understand
Easier to Maintain	Update code in one place
Simplifies Debugging	Easier to find and fix errors
Modular Programming	Breaks large programs into smaller parts
Saves Time	Speeds up program development

Key Points

- Functions make programs reusable and organized.
- They reduce code duplication.
- They improve readability and maintainability.
- They simplify debugging.
- Functions are an essential part of writing clean and efficient Python programs.

Types of Functions

In Python, functions are mainly divided into **two types**:

1. Built-in Functions
2. User-defined Functions

---

1. Built-in Functions

Built-in functions are **predefined functions** provided by Python. You can use them directly without creating them.

Examples

- `print()` – Displays output.
- `input()` – Takes input from the user.
- `len()` – Returns the length of an object.
- `type()` – Returns the data type.
- `max()` – Returns the largest value.
- `min()` – Returns the smallest value.
- `sum()` – Returns the sum of values.

Example

In [ ]:
```
numbers = [10, 20, 30, 40]

print(len(numbers))
```

```
print(max(numbers))
print(min(numbers))
print(sum(numbers))
```

Output

```
4
40
10
100
```

Explanation

These functions are already available in Python, so you can use them without defining them.

---

2. User-defined Functions

User-defined functions are functions **created by the programmer** to perform a specific task.

They are created using the `def` keyword.

Example

In [ ]:
```
def greet():
    print("Welcome to Python!")

greet()
```

Output

```
Welcome to Python!
```

Explanation

The function `greet()` is created by the programmer and executed by calling it.

Difference Between Built-in and User-defined Functions

Built-in Functions	User-defined Functions
Already provided by Python.	Created by the programmer.
No need to define them.	Must be defined before use.
Examples: `print()`, `len()`, `type()`	Examples: `greet()`, `add()`, `display()`

Key Points

- Python has two main types of functions.
- **Built-in functions** are provided by Python.
- **User-defined functions** are created using the `def` keyword.
- User-defined functions allow you to write reusable code for specific tasks.

Defining a Function

A **function** is defined using the `def` keyword followed by the function name and parentheses `()`.

The code inside the function is called the **function body** and must be properly indented.

Defining a function only creates it. The function will not execute until it is called.

Syntax

```
def function_name():
    # Function body
```

Syntax Explanation

- `def` → Keyword used to define a function.
- `function_name` → Name of the function.
- `()` → Parentheses that can contain parameters.
- `:` → Marks the beginning of the function body.
- Indented block → Statements that belong to the function.

Example 1: Define a Simple Function

In [ ]:
```
def greet():
    print("Welcome to Python!")
```

Output

```
No Output
```

Explanation

The function `greet()` is created, but it is **not executed** because it has not been called.

Example 2: Define Another Function

In [ ]:
```
def show_message():
    print("Learning Python is fun!")
```

Output

```
No Output
```

Explanation

The function `show_message()` is defined successfully. It will execute only when it is called.

Example 3: Define Multiple Functions

In [ ]:
```
def greet():
    print("Hello!")

def goodbye():
    print("Goodbye!")
```

Output

```
No Output
```

Explanation

Both functions are defined, but neither one runs until it is called.

Rules for Function Names

- Function names should be meaningful.
- They can contain letters, numbers, and underscores (`_`).
- They cannot start with a number.
- They cannot contain spaces.
- They should follow the **snake_case** naming convention.

Valid Function Names

```
calculate_sum()
display_result()
print_message()
```

Invalid Function Names

```
2sum()
my function()
```

Key Points

- Functions are defined using the `def` keyword.
- A function must have a name followed by parentheses `()`.
- The function body must be indented.
- Defining a function does not execute it.
- A function executes only when it is called.

Calling a Function

After defining a function, you need to **call** it to execute its code.

A function is called by writing its **name followed by parentheses** `()`.

If a function is never called, the code inside it will not run.

Syntax

```
function_name()
```

Example 1: Calling a Function

In [ ]:
```
def greet():
    print("Welcome to Python!")

greet()
```

Output

```
Welcome to Python!
```

Explanation

- The function `greet()` is defined.
- `greet()` is called.
- The code inside the function is executed.

Example 2: Calling a Function Multiple Times

In [ ]:
```
def greet():
    print("Hello!")

greet()
greet()
greet()
```

Output

```
Hello!
Hello!
Hello!
```

Explanation

A function can be called as many times as needed without rewriting the same code.

Example 3: Calling Multiple Functions

In [ ]:
```
def greet():
    print("Good Morning!")

def goodbye():
    print("Goodbye!")

greet()
goodbye()
```

Output

```
Good Morning!
Goodbye!
```

Explanation

Each function performs its own task and executes only when it is called.

Example 4: Calling a Function Inside Another Function

In [ ]:
```python
def greet():
    print("Hello!")

def welcome():
    greet()
    print("Welcome to Python!")

welcome()
```

Output

```
Hello!
Welcome to Python!
```

Explanation

The function `welcome()` calls another function (`greet()`) before printing its own message.

Key Points

- A function must be **called** to execute its code.
- Call a function using its **name followed by parentheses** `()` .
- A function can be called multiple times.
- One function can call another function.
- If a function is not called, its code will not execute.

Function Arguments

An **argument** is a value that is passed to a function when it is called.

Arguments provide data to the function so it can perform a specific task.

The values passed as arguments are received by the function through **parameters**.

Syntax

```
function_name(argument1, argument2, ...)
```

Example 1: One Argument

In [ ]:
```python
def greet(name):
    print("Hello,", name)

greet("Alice")
```

Output

```
Hello, Alice
```

Explanation

- `name` is the **parameter**.
- `"Alice"` is the **argument** passed to the function.

Example 2: Multiple Arguments

In [ ]:
```python
def add(a, b):
    print(a + b)

add(10, 20)
```

Output

```
30
```

Explanation

- `a` and `b` are **parameters**.
- `10` and `20` are **arguments**.

Parameters vs Arguments

Parameters	Arguments
Variables defined in the function definition.	Values passed when calling the function.
Receive data from the caller.	Provide data to the function.
Example: `name`	Example: `"Alice"`

Example

In [ ]:
```python
def greet(name):      # name → Parameter
    print("Hello,", name)

greet("Alice")        # "Alice" → Argument
```

Key Points

- Arguments are values passed to a function.
- Parameters receive the values passed as arguments.
- A function can accept one or more arguments.
- The number of arguments should match the number of parameters unless using default values or special argument types.

Positional Arguments

Positional arguments are arguments that are passed to a function **in the same order** as the parameters are defined.

Python matches each argument to its corresponding parameter based on its position.

> **Note:** The order of positional arguments is important.

Syntax

```
def function_name(parameter1, parameter2):
    # Code

function_name(argument1, argument2)
```

Example 1: Positional Arguments

In [ ]:
```
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student("Alice", 20)
```

Output

```
Name: Alice
Age: 20
```

Explanation

- `"Alice"` is assigned to `name`.
- `20` is assigned to `age`.
- The arguments match the parameters by their position.

Example 2: Multiple Positional Arguments

In [ ]:
```
def add(a, b):
    print(a + b)

add(10, 20)
```

Output

```
30
```

Explanation

- `10` is assigned to `a`.
- `20` is assigned to `b`.
- The function adds the two values.

Example 3: Incorrect Order

In [ ]:
```
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student(20, "Alice")
```

Output

```
Name: 20
Age: Alice
```

Explanation

The arguments are matched by **position**, not by their meaning. Since the order is incorrect, the values are assigned to the wrong parameters.

Key Points

- Positional arguments are matched by their order.
- The order of arguments must match the order of parameters.
- Passing arguments in the wrong order may produce incorrect results.
- Positional arguments are the most commonly used type of function arguments.

Keyword Arguments

Keyword arguments are arguments passed to a function by specifying the **parameter name** followed by the value.

With keyword arguments, **the order of the arguments does not matter** because Python matches them using their parameter names.

> **Note:** Keyword arguments make your code more readable and flexible.

Syntax

```
def function_name(parameter1, parameter2):
    # Code

function_name(parameter1=value1, parameter2=value2)
```

Example 1: Keyword Arguments

In [ ]:
```
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student(name="Alice", age=20)
```

Output

```
Name: Alice
Age: 20
```

Explanation

The arguments are passed using the parameter names, so Python assigns the values correctly.

Example 2: Changing the Order

In [ ]:
```
def student(name, age):
    print("Name:", name)
    print("Age:", age)
```

```
student(age=20, name="Alice")
```

Output

```
Name: Alice
Age: 20
```

Explanation

Even though the order is changed, the output remains the same because the arguments are matched by their parameter names.

Example 3: Mixing Positional and Keyword Arguments

In [ ]:
```
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student("Alice", age=20)
```

Output

```
Name: Alice
Age: 20
```

Explanation

The first argument is passed by position, and the second is passed as a keyword argument.

> **Note:** Positional arguments must always come before keyword arguments.

Incorrect Example

In [ ]:
```
def student(name, age):
    print(name, age)

student(name="Alice", 20)
```

Result

```
SyntaxError
```

Explanation

A positional argument cannot come after a keyword argument.

Key Points

- Keyword arguments are passed using **parameter names**.
- The order of keyword arguments does not matter.
- Keyword arguments improve code readability.
- Positional arguments must come before keyword arguments when both are used.

Default Arguments

A **default argument** is a parameter that has a **default value** assigned to it.

If no argument is passed for that parameter when the function is called, Python automatically uses the default value.

Default arguments make some function arguments **optional**.

Syntax

```
def function_name(parameter1, parameter2):
    # Code

function_name(parameter1=value1, parameter2=value2)
```

Example 1: Using a Default Argument

In [ ]:
```
def greet(name="Guest"):
    print("Hello,", name)

greet()
```

Output

```
Hello, Guest
```

Explanation

Since no argument is passed, Python uses the default value `"Guest"`.

Example 2: Passing an Argument

In [ ]:
```
def greet(name="Guest"):
    print("Hello,", name)

greet("Alice")
```

Output

```
Hello, Alice
```

Explanation

The argument `"Alice"` replaces the default value `"Guest"`.

Example 3: Multiple Default Arguments

In [ ]:
```
def student(name, age=18):
    print("Name:", name)
    print("Age:", age)

student("Alice")
student("Bob", 20)
```

Output

```
Name: Alice
Age: 18

Name: Bob
Age: 20
```

Explanation

- In the first function call, only the `name` argument is provided, so `age` uses its default value of `18`.
- In the second function call, both arguments are provided, so the default value is replaced.

Rules for Default Arguments

- Parameters with default values should be placed **after** parameters without default values.

Correct

In [ ]:
```python
def student(name, age=18):
    print(name, age)
```

Incorrect

In [ ]:
```python
def student(age=18, name):
    print(name, age)
```

Result

```
SyntaxError
```

Key Points

- A default argument has a predefined value.
- If no argument is passed, the default value is used.
- If an argument is passed, it replaces the default value.
- Parameters with default values should come after non-default parameters.

Arbitrary Arguments (`*args`)

Sometimes you may not know **how many arguments** will be passed to a function.

In such cases, Python provides **Arbitrary Arguments** using `*args`.

The `*args` parameter allows a function to accept **any number of positional arguments**.

All the arguments are stored as a **tuple**.

> **Note:** The name `args` is a convention. You can use any name, but the `*` is required.

Syntax

```
def function_name(*args):
    # Code
```

Example 1: Accepting Multiple Arguments

In [ ]:
```
def fruits(*args):
    print(args)

fruits("Apple", "Banana", "Mango")
```

Output

```
('Apple', 'Banana', 'Mango')
```

Explanation

- The function accepts three arguments.
- All arguments are stored in a tuple named `args`.

Example 2: Accessing Individual Values

In [ ]:
```
def fruits(*args):
    print(args[0])
    print(args[1])

fruits("Apple", "Banana", "Mango")
```

Output

```
Apple
Banana
```

Explanation

Since `args` is a tuple, you can access its elements using indexing.

Example 3: Using a Loop

In [ ]:
```
def numbers(*args):
    for num in args:
        print(num)

numbers(10, 20, 30, 40)
```

Output

```
10
20
30
40
```

Explanation

The `for` loop iterates through every value stored in the `args` tuple.

Example 4: Fixed Parameter with `*args`

In [ ]:
```python
def student(name, *marks):
    print("Name:", name)
    print("Marks:", marks)

student("Alice", 90, 85, 95)
```

Output

```
Name: Alice
Marks: (90, 85, 95)
```

Explanation

- `"Alice"` is assigned to `name`.
- The remaining values are stored in the tuple `marks`.

Rules for `*args`

- `*args` accepts any number of positional arguments.
- The arguments are stored as a tuple.
- You can iterate over `args` using a loop.
- If a function has normal parameters and `*args`, the normal parameters come first.

Key Points

- Use `*args` when the number of positional arguments is unknown.
- `*args` stores all extra arguments in a tuple.
- You can access tuple elements using indexing or loops.
- The `*` is required; the name `args` is just a convention.

Arbitrary Keyword Arguments (`**kwargs`)

Sometimes you may not know **how many keyword arguments** will be passed to a function.

In such cases, Python provides **Arbitrary Keyword Arguments** using `**kwargs`.

The `**kwargs` parameter allows a function to accept **any number of keyword arguments**.

All the keyword arguments are stored as a **dictionary**.

> **Note:** The name `kwargs` is a convention. You can use any valid name, but the `**` is required.

Syntax

```python
def function_name(**kwargs):
    # Code
```

Example 1: Accepting Keyword Arguments

In [ ]:
```python
def student(**kwargs):
    print(kwargs)

student(name="Alice", age=20, city="Delhi")
```

Output

```
{'name': 'Alice', 'age': 20, 'city': 'Delhi'}
```

Explanation

- The function accepts three keyword arguments.
- All the arguments are stored in a dictionary named `kwargs`.

Example 2: Accessing Individual Values

In [ ]:
```python
def student(**kwargs):
    print("Name:", kwargs["name"])
    print("Age:", kwargs["age"])

student(name="Alice", age=20)
```

Output

```
Name: Alice
Age: 20
```

Explanation

Since `kwargs` is a dictionary, values are accessed using their keys.

Example 3: Using a Loop

In [ ]:
```python
def student(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)

student(name="Alice", age=20, city="Delhi")
```

Output

```
name : Alice
age : 20
city : Delhi
```

Explanation

The `items()` method returns both the key and value from the dictionary, allowing you to iterate through all keyword arguments.

Example 4: Fixed Parameter with `**kwargs`

In [ ]:
```python
def student(course, **details):
    print("Course:", course)
    print(details)
```

```
student("Python", name="Alice", age=20)
```

Output

```
Course: Python
{'name': 'Alice', 'age': 20}
```

Explanation

- `"Python"` is assigned to the `course` parameter.
- The remaining keyword arguments are stored in the dictionary `details`.

Difference Between `*args` and `**kwargs`

`*args`	`**kwargs`
Accepts any number of positional arguments.	Accepts any number of keyword arguments.
Stores values in a tuple.	Stores values in a dictionary.
Access values using indexes.	Access values using keys.

Rules for `**kwargs`

- `**kwargs` accepts any number of keyword arguments.
- The arguments are stored as a dictionary.
- Keys must be unique.
- If a function has normal parameters and `**kwargs`, the normal parameters come first.

Key Points

- Use `**kwargs` when the number of keyword arguments is unknown.
- `**kwargs` stores all keyword arguments in a dictionary.
- Dictionary values can be accessed using keys.
- The `**` is required; the name `kwargs` is just a convention.

Return Statement

The `return` statement is used to send a value back from a function to the place where the function was called.

Unlike `print()`, which only displays output on the screen, `return` gives the result back so it can be stored in a variable, used in calculations, or passed to another function.

> **Note:** When a `return` statement is executed, the function immediately stops executing.

Syntax

```
def function_name():
    # Code
    return value
```

Example 1: Returning a Value

In [ ]:
```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Output

```
30
```

Explanation

- The function adds `10` and `20`.
- The result `30` is returned.
- The returned value is stored in the variable `result`.

Example 2: Returning a String

In [ ]:
```
def greet(name):
    return "Hello, " + name

message = greet("Alice")
print(message)
```

Output

```
Hello, Alice
```

Explanation

The function returns a greeting instead of printing it directly.

Example 3: Returning Multiple Values

In [ ]:
```
def calculate(a, b):
    return a + b, a - b

sum_value, difference = calculate(10, 5)

print(sum_value)
print(difference)
```

Output

```
15
5
```

Explanation

Python allows a function to return multiple values separated by commas. These values can be stored in separate variables.

Example 4: Returning Early

In [ ]:
```python
def check_age(age):
    if age < 18:
        return "Not Eligible"
    return "Eligible"

print(check_age(16))
print(check_age(20))
```

Output

```
Not Eligible
Eligible
```

Explanation

- If `age` is less than `18`, the function returns `"Not Eligible"` immediately.
- Otherwise, it continues and returns `"Eligible"`.

Difference Between `print()` and `return`

`print()`	`return`
Displays output on the screen.	Sends a value back to the caller.
Cannot be reused later.	Returned value can be stored and reused.
Mainly used for displaying information.	Used when a function needs to produce a result.

Example

In [ ]:
```python
def add1(a, b):
    print(a + b)

def add2(a, b):
    return a + b

x = add2(10, 20)
print(x)
```

Output

```
30
```

When to Use `return`

Use `return` when you want to:

- Send a result back from a function.
- Store the result in a variable.
- Use the result in another calculation.
- Reuse the returned value elsewhere in the program.

Key Points

- `return` sends a value back from a function.
- A function can return one or more values.
- Once `return` is executed, the function stops.
- Returned values can be stored, reused, or passed to other functions.
- `return` is generally preferred over `print()` when a function needs to produce a result.

Scope of Variables

The **scope of a variable** determines **where a variable can be accessed** in a program.

In Python, a variable may be available only inside a function or throughout the entire program, depending on where it is created.

There are two main types of variable scope:

1. Local Scope
2. Global Scope

Why is Variable Scope Important?

Variable scope helps:

- Control where variables can be used.
- Prevent accidental changes to variables.
- Organize code more effectively.
- Avoid naming conflicts.

Types of Variable Scope

1. Local Scope

A variable created **inside a function** is called a **local variable**.

It can only be accessed inside that function.

---

2. Global Scope

A variable created **outside all functions** is called a **global variable**.

It can be accessed from anywhere in the program.

Real-Life Example

Imagine a classroom.

- A student's notebook belongs only to that student.

    - This is like a **local variable**.
- The classroom whiteboard can be seen by everyone.
    - This is like a **global variable**.

Key Points

- Variable scope defines where a variable can be accessed.
- Variables can have either local scope or global scope.
- Local variables exist only inside a function.
- Global variables can be accessed throughout the program.

Local Variables

A **local variable** is a variable that is created **inside a function**.

It can only be accessed within that function and is destroyed when the function finishes executing.

> **Note:** Local variables are not available outside the function in which they are created.

Syntax

```
def function_name():
    variable_name = value
```

Example 1: Local Variable

In [ ]:
```
def greet():
    message = "Welcome to Python!"
    print(message)

greet()
```

Output

```
Welcome to Python!
```

Explanation

- `message` is created inside the `greet()` function.
- It is a local variable.
- It can only be accessed within the `greet()` function.

Example 2: Accessing a Local Variable Outside the Function

In [ ]:
```python
def greet():
    message = "Welcome to Python!"

greet()

print(message)
```

Output

```
NameError: name 'message' is not defined
```

Explanation

The variable `message` exists only inside the `greet()` function. Trying to access it outside the function causes a `NameError`.

Example 3: Local Variables in Different Functions

In [ ]:
```python
def student():
    name = "Alice"
    print(name)

def teacher():
    name = "John"
    print(name)

student()
teacher()
```

Output

```
Alice
John
```

Explanation

- Both functions have a variable named `name`.
- These are different local variables.
- They do not affect each other because each belongs to its own function.

Characteristics of Local Variables

- Created inside a function.
- Accessible only within that function.
- Created when the function is called.
- Destroyed when the function finishes execution.
- Different functions can have local variables with the same name.

Key Points

- A local variable is declared inside a function.
- It cannot be accessed outside the function.
- Local variables exist only during the execution of the function.
- Using local variables helps keep functions independent and prevents accidental changes to data.

Global Variables

A **global variable** is a variable that is created **outside all functions**.

Unlike a local variable, a global variable can be accessed from **anywhere in the program**, including inside functions.

> **Note:** A global variable is available throughout the program unless it is modified or deleted.

Syntax

```
variable_name = value

def function_name():
    print(variable_name)
```

Example 1: Accessing a Global Variable

In [ ]:
```
message = "Welcome to Python!"

def greet():
    print(message)

greet()
```

Output

```
Welcome to Python!
```

Explanation

- `message` is declared outside the function.
- It is a global variable.
- The function can access and display its value.

Example 2: Accessing a Global Variable Inside and Outside a Function

In [ ]:
```
language = "Python"

def show_language():
    print("Inside Function:", language)

show_language()

print("Outside Function:", language)
```

Output

```
Inside Function: Python
Outside Function: Python
```

Explanation

The global variable `language` can be accessed both inside and outside the function.

Example 3: Local Variable with the Same Name

In [ ]:
```
name = "Alice"

def display():
    name = "Bob"
    print("Inside Function:", name)

display()

print("Outside Function:", name)
```

Output

```
Inside Function: Bob
Outside Function: Alice
```

Explanation

- The global variable is `name = "Alice"` .
- Inside the function, another variable named `name` is created.
- This local variable hides the global variable inside the function.
- The global variable remains unchanged outside the function.

Characteristics of Global Variables

- Declared outside all functions.
- Accessible throughout the program.
- Can be read inside functions.
- Can have the same name as a local variable.
- Remain in memory until the program finishes.

Key Points

- A global variable is declared outside all functions.
- It can be accessed inside and outside functions.
- A local variable with the same name temporarily hides the global variable inside that function.
- To **modify** a global variable inside a function, use the `global` keyword (covered next).

The `global` Keyword

Normally, a function can **read** a global variable, but it **cannot modify** it directly.

If you want to modify a global variable inside a function, you must use the `global` keyword.

The `global` keyword tells Python that the variable refers to the global variable, not a new local variable.

Syntax

```
global variable_name
```

Example 1: Modifying a Global Variable

In [ ]:
```
count = 10

def update():
    global count
    count = 20

update()

print(count)
```

Output

```
20
```

Explanation

- `count` is a global variable.
- The `global` keyword allows the function to modify its value.
- After calling `update()`, the value of `count` becomes `20`.

Example 2: Without the `global` Keyword

In [ ]:
```
count = 10

def update():
    count = 20
    print("Inside Function:", count)

update()

print("Outside Function:", count)
```

Output

```
Inside Function: 20
Outside Function: 10
```

Explanation

- The variable `count` inside the function is a **local variable**.
- It does not affect the global variable.
- The global variable remains unchanged.

Example 3: Creating a Global Variable Inside a Function

In [ ]:
```
def create():
    global message
    message = "Hello, Python!"

create()
```

```
print(message)
```

Output

```
Hello, Python!
```

Explanation

The `global` keyword creates the variable `message` as a global variable, making it accessible outside the function.

Without `global`

In [ ]:
```
x = 5

def test():
    x = 10
    print("Inside:", x)

test()

print("Outside:", x)
```

Output

```
Inside: 10
Outside: 5
```

The variable inside the function is local, so the global variable is not changed.

With `global`

In [ ]:
```
x = 5

def test():
    global x
    x = 10
    print("Inside:", x)

test()

print("Outside:", x)
```

Output

```
Inside: 10
Outside: 10
```

The `global` keyword allows the function to modify the global variable.

When Should You Use `global`?

Use the `global` keyword only when you need to modify a global variable inside a function.

In most programs, it is better to pass values as function arguments and return results instead of relying on global variables. This makes your code easier to understand and maintain.

Key Points

- The `global` keyword is used to modify a global variable inside a function.
- Without `global`, assigning a value inside a function creates a new local variable.
- `global` can also be used to create a global variable from inside a function.
- Use `global` only when necessary, as excessive use can make programs harder to maintain.

Recursion (Introduction)

Recursion is a programming technique in which a **function calls itself** to solve a problem.

A recursive function keeps calling itself until a **base case** is reached.

> **Note:** Every recursive function must have a **base case**. Without it, the function will keep calling itself forever and cause a `RecursionError`.

How Recursion Works

A recursive function has two parts:

1. **Base Case** – Stops the recursion.
2. **Recursive Case** – Calls the function again.

Syntax

```
def function_name():
    if base_case:
        return value
    return function_name()
```

Example 1: Countdown

In [ ]:
```
def countdown(n):
    if n == 0:
        print("Done!")
        return

    print(n)
    countdown(n - 1)

countdown(5)
```

Output

```
5
4
3
2
1
Done!
```

Explanation

- The function prints the current value of `n`.
- It calls itself with `n - 1`.
- When `n` becomes `0`, the base case stops the recursion.

Example 2: Factorial

In [ ]:
```python
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

Output

```
120
```

Explanation

The function multiplies each number by the factorial of the previous number until it reaches the base case (`n == 1`).

Calculation:

```
factorial(5)
= 5 × factorial(4)
= 5 × 4 × factorial(3)
= 5 × 4 × 3 × factorial(2)
= 5 × 4 × 3 × 2 × factorial(1)
= 5 × 4 × 3 × 2 × 1
= 120
```

Advantages of Recursion

- Makes some problems easier to solve.
- Produces shorter and cleaner code for recursive problems.
- Useful for tree and graph algorithms.
- Commonly used in searching and backtracking algorithms.

Disadvantages of Recursion

- Can be slower than loops.
- Uses more memory because each function call is stored.
- Missing a base case causes a `RecursionError`.

Key Points

- Recursion is when a function calls itself.
- Every recursive function must have a **base case**.
- The base case stops the recursion.

- Without a base case, recursion continues until a `RecursionError` occurs.
- For many simple problems, loops are often easier to understand than recursion.

Lambda Functions (Introduction)

A **Lambda Function** is a small, anonymous function created using the `lambda` keyword.

Unlike a normal function, a lambda function does **not require a name** and is usually written in a single line.

Lambda functions are useful for short and simple operations.

> **Note:** A lambda function can have any number of parameters, but it can contain only **one expression**.

Syntax

```
lambda parameters: expression
```

Syntax Explanation

- `lambda` → Keyword used to create a lambda function.
- `parameters` → Input values for the function.
- `expression` → A single expression whose result is automatically returned.

Example 1: Lambda Function

In [ ]:
```
square = lambda x: x * x

print(square(5))
```

Output

```
25
```

Explanation

- `x` is the parameter.
- `x * x` is the expression.
- Calling `square(5)` returns `25`.

Example 2: Multiple Parameters

In [ ]:
```
add = lambda a, b: a + b

print(add(10, 20))
```

Output

```
30
```

Explanation

The lambda function accepts two parameters and returns their sum.

Example 3: Lambda vs Normal Function

Normal Function

In [ ]:
```
def multiply(a, b):
    return a * b

print(multiply(4, 5))
```

Lambda Function

In [ ]:
```
multiply = lambda a, b: a * b

print(multiply(4, 5))
```

Output

```
20
20
```

Explanation

Both functions produce the same result. The lambda version is shorter and is commonly used for simple operations.

When Should You Use Lambda Functions?

Use lambda functions when:

- The function is small and simple.
- The function is needed only once.
- You want to write short, readable code.

For larger or more complex tasks, use a normal function defined with the `def` keyword.

Difference Between Normal Functions and Lambda Functions

Normal Function	Lambda Function
Created using the `def` keyword.	Created using the `lambda` keyword.
Has a function name.	Usually anonymous (no name).
Can contain multiple statements.	Can contain only one expression.
Suitable for large programs.	Suitable for short, simple tasks.

Key Points

- Lambda functions are anonymous functions.
- They are created using the `lambda` keyword.
- A lambda function can have multiple parameters.
- It can contain only one expression.
- Lambda functions are best for short and simple operations.

Common Errors

- Forgetting to call the function.
- Incorrect indentation inside a function.
- Passing the wrong number of arguments.
- Forgetting to use the `return` statement.
- Accessing a local variable outside the function.
- Modifying a global variable without using `global`.
- Missing the base case in recursion.
- Confusing `print()` with `return`.

Best Practices

- Use meaningful function names.
- Keep functions short and focused on one task.
- Use parameters instead of global variables whenever possible.
- Return values instead of printing inside functions when appropriate.
- Follow proper indentation.
- Add comments or docstrings for complex functions.
- Reuse functions instead of writing duplicate code.
- Use `lambda` functions only for simple, one-line operations.

Coding Questions

- 1. Write a function to print "Hello, World!".
- 2. Write a function that takes a number as an argument and prints its square.
- 3. Write a function that takes two numbers and returns their sum.
- 4. Write a function that accepts a student's name and age, then displays them.
- 5. Write a function that checks whether a number is even or odd.
- 6. Write a function that accepts three numbers and returns the largest number.
- 7. Write a function that calculates the factorial of a number using recursion.
- 8. Write a function that accepts any number of integers using *args and returns their sum.
- 9. Write a function that accepts student details using **kwargs and displays each key-value pair.
- 10. Write a function that returns both the quotient and remainder of two numbers.

💻 Practice Programs (10)

- 1. Write a function that takes the length and width as arguments and returns the area of a rectangle.
- 2. Write a function that converts a temperature from Celsius to Fahrenheit.
- 3. Write a function that counts the number of vowels (`a, e, i, o, u`) in a given string.
- 4. Write a function that checks whether a given string is a palindrome.
- 5. Write a function that counts the number of uppercase and lowercase letters in a string.
- 6. Write a function that returns the reverse of a given string.
- 7. Write a function that finds and returns the second largest element in a list.
- 8. Write a function that removes duplicate elements from a list and returns the updated list.
- 9. Write a function that counts the frequency of each character in a string and displays the result.
- 10. Create a menu-driven calculator using functions that performs the following operations:
    - Addition
    - Subtraction
    - Multiplication
    - Division

📌 Cheat Sheet

Define a Function

```
def function_name():
    pass
```

Call a Function

```
function_name()
```

Function with Parameters

```
def greet(name):
    print(name)
```

Return Statement

```
def add(a, b):
    return a + b
```

Default Argument

```
def greet(name="Guest"):
    print(name)
```

`*args`

```
def func(*args):
    print(args)
```

`**kwargs`

```python
def func(**kwargs):
    print(kwargs)
```

Local Variable

```python
def demo():
    x = 10
```

Global Variable

```python
x = 10
```

Global Keyword

```python
global x
```

Recursion

```python
def func():
    func()
```

Lambda Function

```python
square = lambda x: x * x
```

📝 Summary

In this lesson, you learned:

- Functions and their importance.
- Defining and calling functions.
- Parameters and arguments.
- Positional, keyword, default, `*args`, and `**kwargs` arguments.
- The `return` statement.
- Local and global variables.
- The `global` keyword.
- Basic concepts of recursion.
- Introduction to lambda functions.

Functions help make programs **reusable, organized, and easier to maintain**. Mastering functions is an important step toward writing clean and efficient Python programs.